

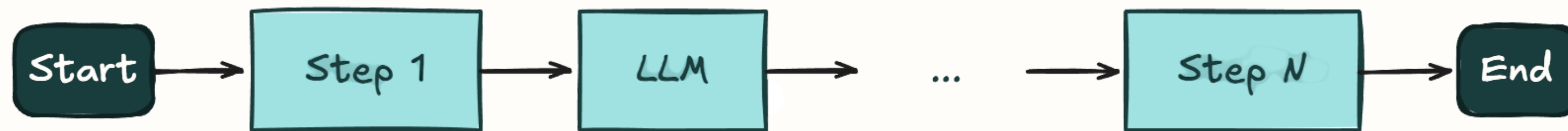
Introduction to **Middleware**

A large language model alone mainly predicts text ...



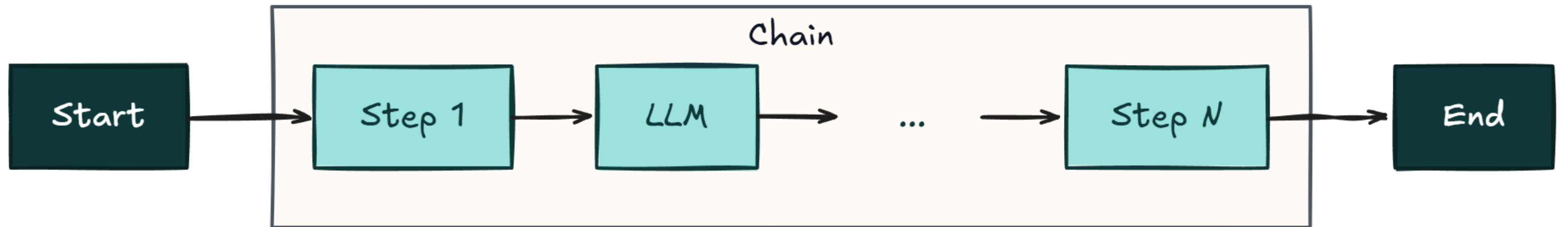
... it can't access things like outside tools or external content.

So, many LLM applications use **control flows** ...



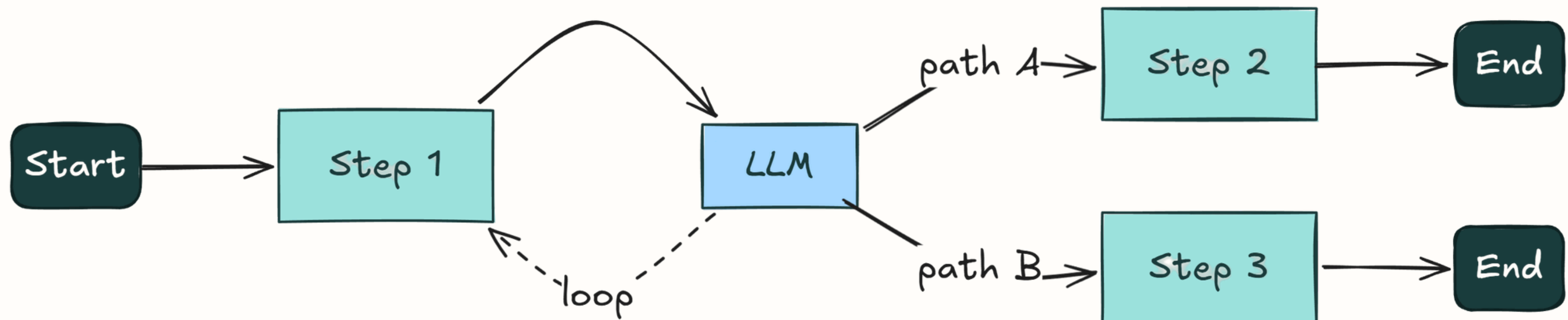
... to give LLMs more capabilities (tool calls, context, etc.)

This flow creates a **chain** ...



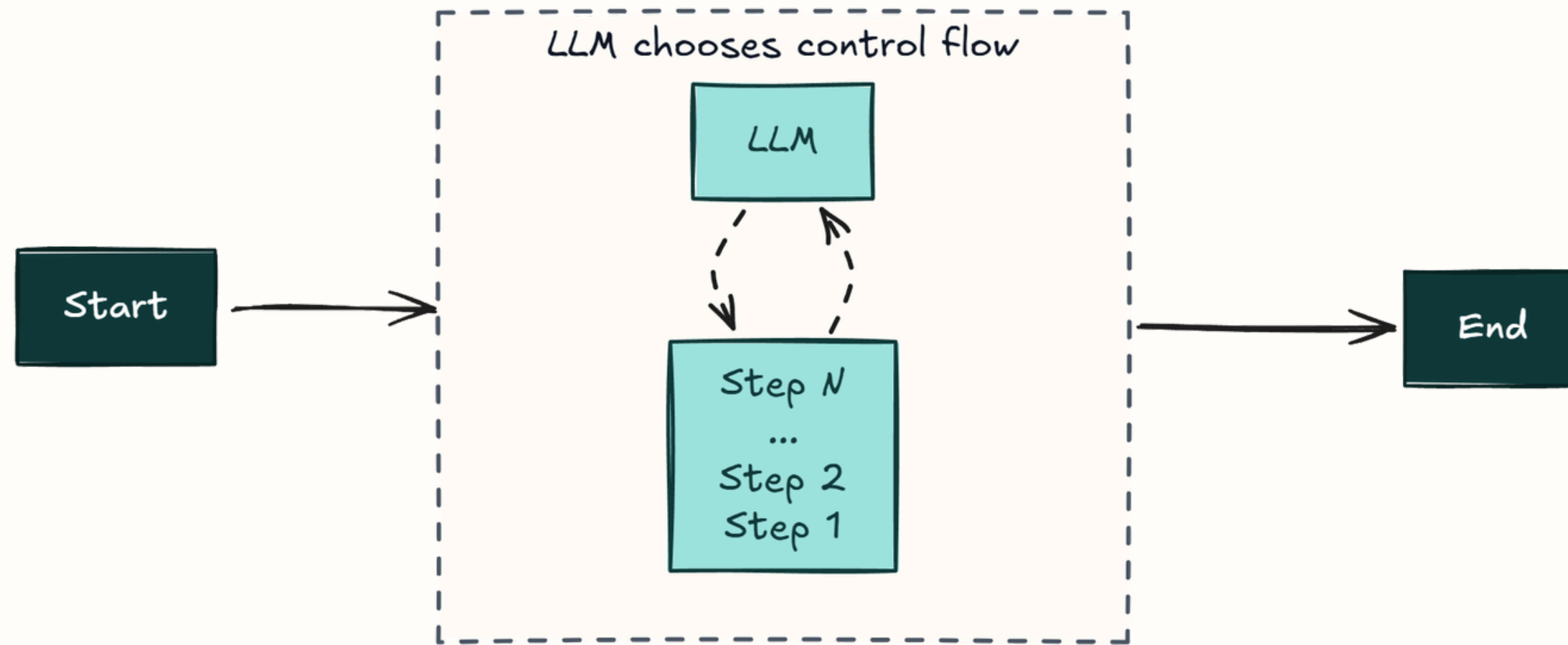
... these chains are reliable and can be **routed**.

Agent is a control flow defined by an LLM ...



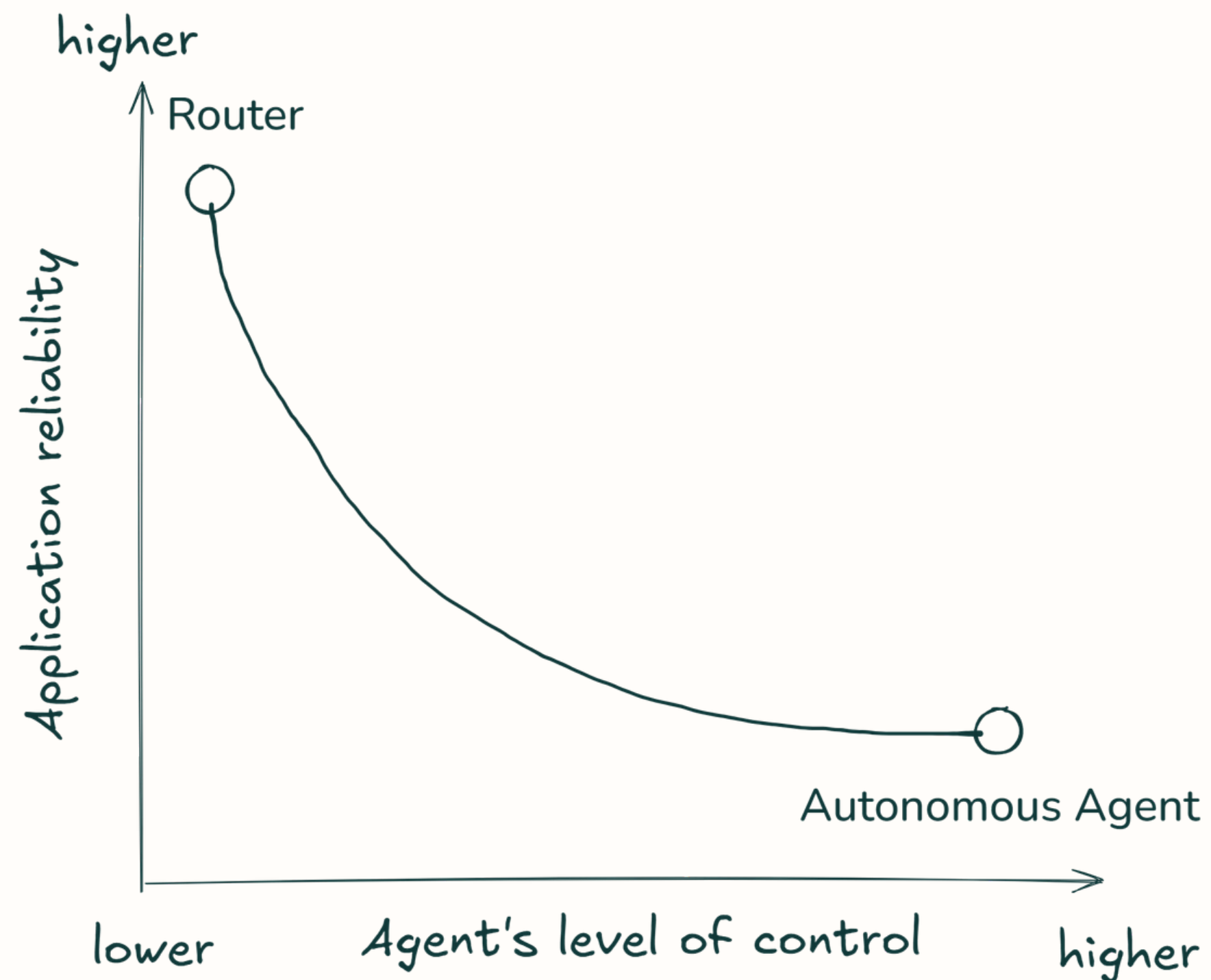
... where model chooses the next action.

LLM systems that can pick their own control flow ...

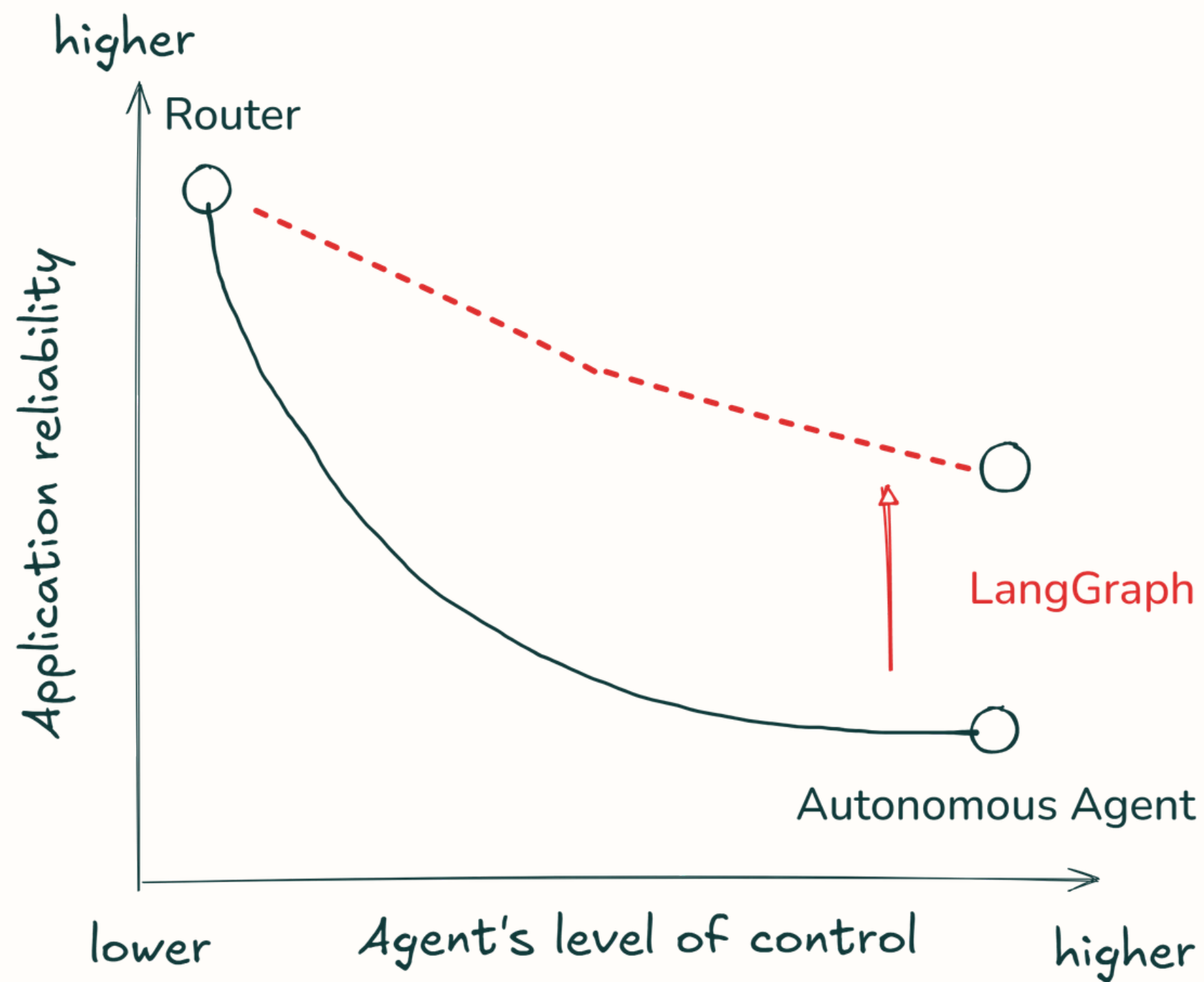


... are usually more capable.

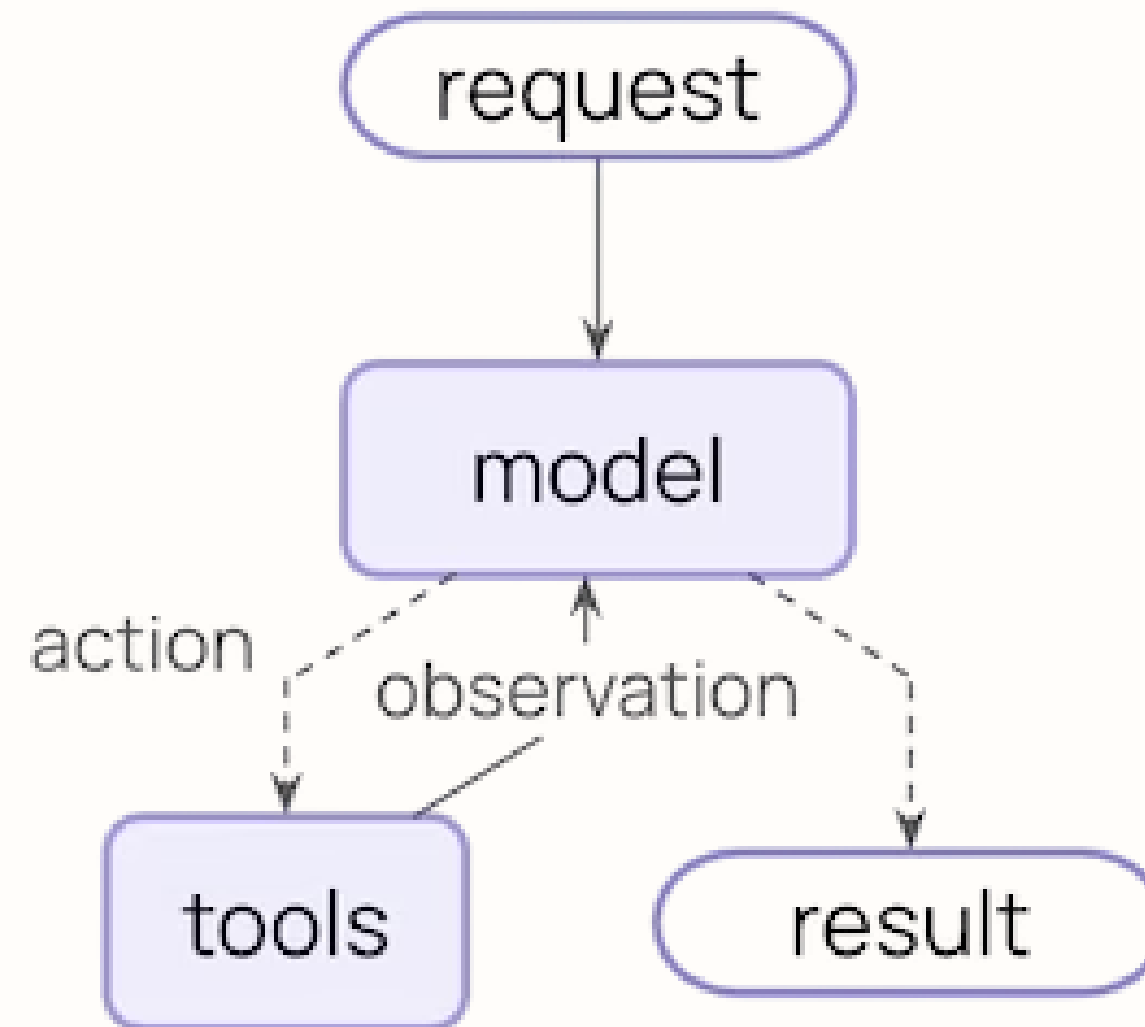
LangGraph aims balance reliability with control



LangGraph aims balance reliability with control

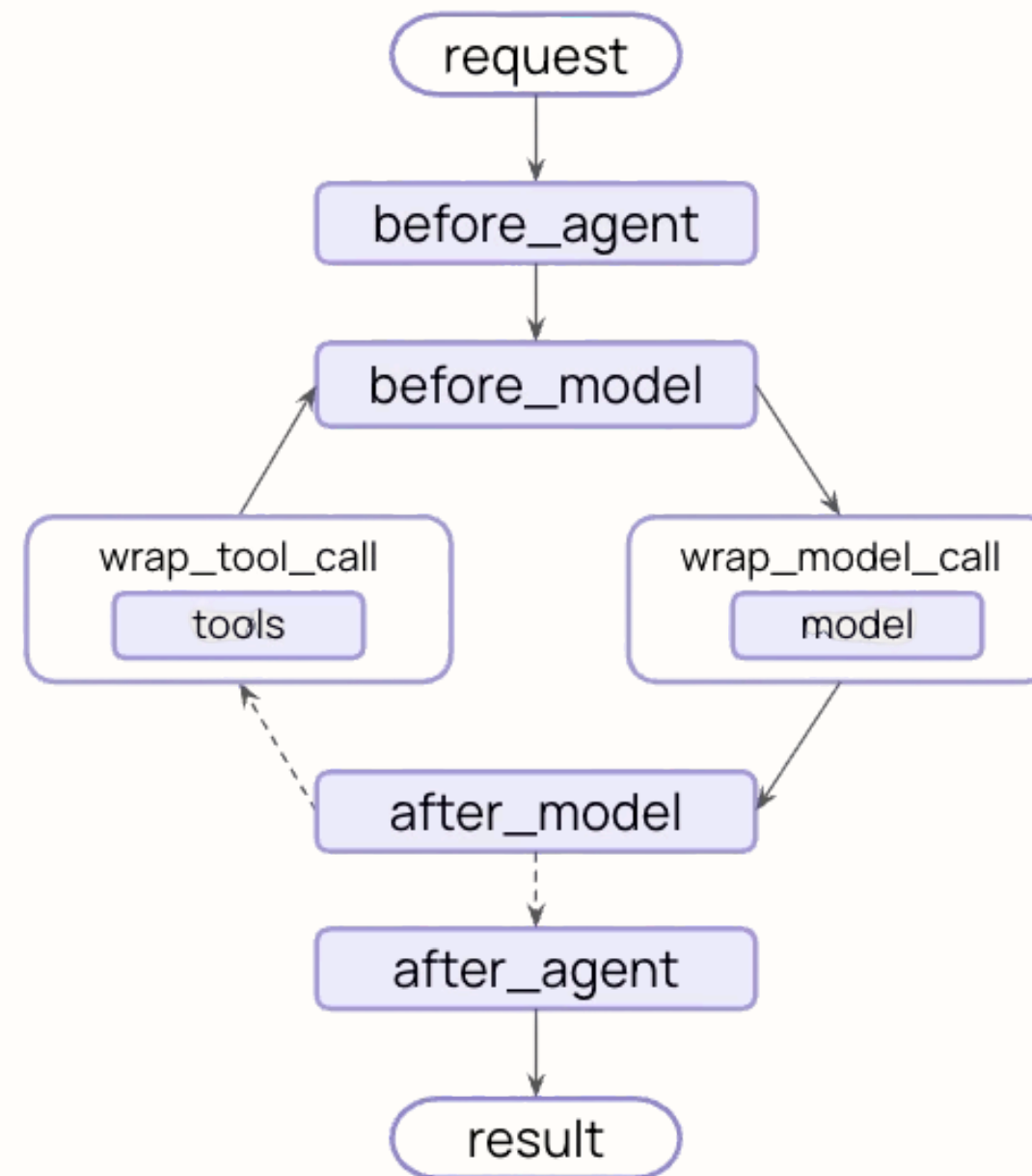


The core agent loop = ReAct



Act, observe, and reason until outcome is reached.

Middleware is a way to add behavior around the agent loop ...



... without rewriting the agent.

Prebuilt middleware

There's prebuilt middleware for common usecases.

Middleware	Description
<u>Summarization</u>	Automatically summarize conversation history when approaching token limits.
<u>Human-in-the-loop</u>	Pause execution for human approval of tool calls.
<u>Model call limit</u>	Limit the number of model calls to prevent excessive costs.
<u>Tool call limit</u>	Control tool execution by limiting call counts.
<u>Model fallback</u>	Automatically fallback to alternative models when primary fails.
<u>PII detection</u>	Detect and handle Personally Identifiable Information (PII).
<u>To-do list</u>	Equip agents with task planning and tracking capabilities.
<u>LLM tool selector</u>	Use an LLM to select relevant tools before calling main model.

You can add custom middleware ...

Node-style hooks run at specific execution points:

Hook	When it runs
<code>before_agent</code>	Before agent starts (once per invocation)
<code>before_model</code>	Before each model call
<code>after_model</code>	After each model response
<code>after_agent</code>	After agent completes (once per invocation)

... at different execution points within the agentic loop.

There are almost infinite possibilities ...

Wrap-style hooks

Intercept execution and control when the handler is called. Use for retries, caching, and transformation.

You decide if the handler is called zero times (short-circuit), once (normal flow), or multiple times (retry logic).

Available hooks:

- `wrap_model_call` - Around each model call
- `wrap_tool_call` - Around each tool call

... which are all more deterministic than prompting alone.

LangSmith

